BÀI TẬP TRÊN LỚP SỐ 2
MÔN: KIẾN TRÚC PHẦN MỀM
ĐỀ TÀI: Thiết kế và triển khai phần mềm quản lý nhà hàng trên nền tảng Web

Nhóm thực hiện: Nhóm 1

1. Cơ sở thực hiện

Tài liệu này được xây dựng dựa trên yêu cầu của đề bài "Bài tập trên lớp số 2", mẫu thu thập yêu cầu nhóm 1 và tài liệu tham chiếu Task 2. Mục tiêu là trình bày một phương án kiến trúc khả thi, có thể triển khai thực tế cho hệ thống quản lý nhà hàng trên nền tảng Web.

Phạm vi hệ thống bao gồm các nghiệp vụ chính: quản lý thực đơn, gọi món, điều phối bếp, thanh toán, quản lý bàn, quản lý khách hàng và thông báo trạng thái đơn hàng theo thời gian thực.

2. Yêu cầu 1a - Lựa chọn mẫu kiến trúc

Nhóm lựa chọn mẫu kiến trúc Microservices (Vi dịch vụ), kết hợp Event-Driven Architecture (Kiến trúc hướng sự kiện).

Lý do lựa chọn:

- Phân tách nghiệp vụ rõ ràng: mỗi service phụ trách một miền nghiệp vụ độc lập (Order, Kitchen, Payment, Notification...).

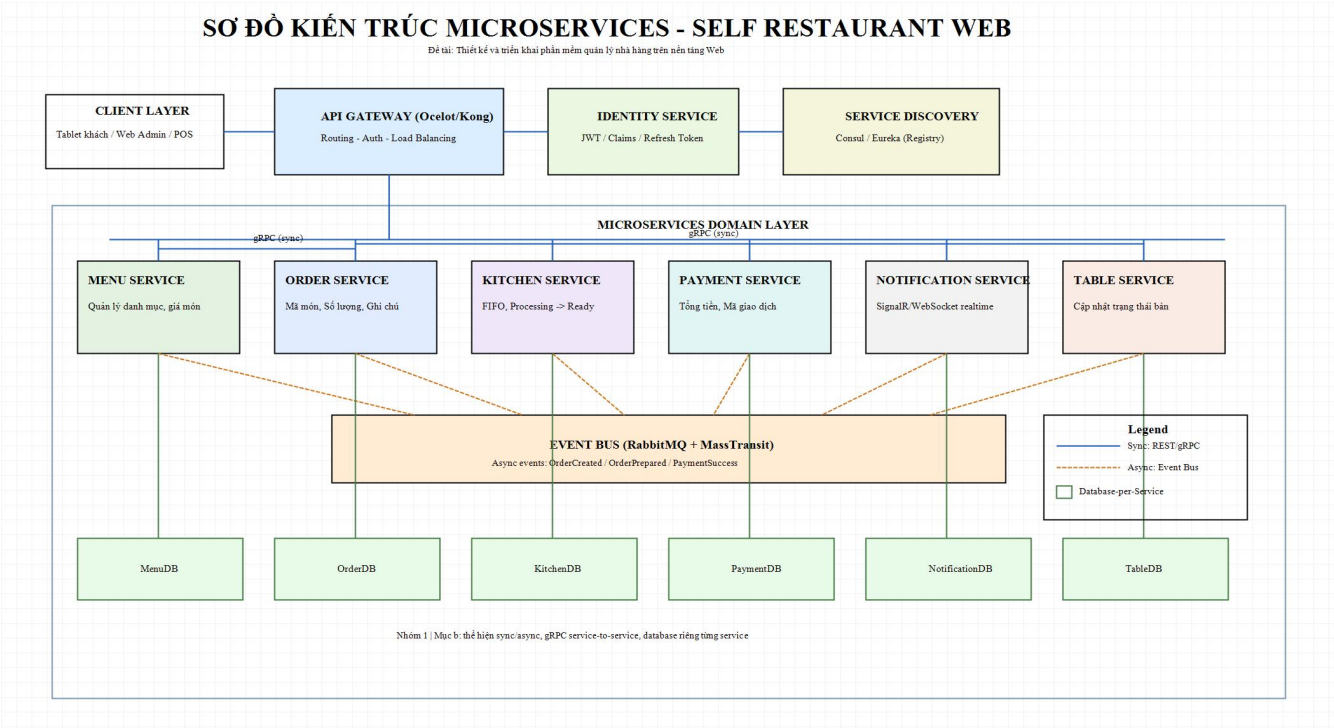
- Khả năng mở rộng tốt: có thể mở rộng riêng các service chịu tải cao vào giờ cao điểm (Order/Kitchen) mà không ảnh hưởng toàn hệ thống.

- Tính sẵn sàng và ổn định: lỗi cục bộ ở một service không làm dừng toàn bộ ứng dụng.

- Hỗ trợ realtime: trạng thái món và đơn hàng được cập nhật tức thời nhờ luồng sự kiện qua Message Broker.

3. Yêu cầu 1b - Biểu đồ mô tả trực quan kiến trúc (vẽ hình)

Biểu đồ kiến trúc tổng quan thể hiện đầy đủ ba khía cạnh: (1) giao tiếp đồng bộ (sync) giữa các service bằng REST/gRPC, (2) giao tiếp bất đồng bộ (async) qua Event Bus RabbitMQ + MassTransit, và (3) mô hình Database-per-Service với cơ sở dữ liệu tách riêng cho từng service.



4. Yêu cầu 1c - Liệt kê và mô tả thành phần (component)

4.1 API Gateway (Ocelot/Kong)

- Chức năng: tiếp nhận request từ client, xác thực ban đầu, định tuyến và cân bằng tải đến các service nội bộ.

- Input: HTTP request từ Web Admin, POS, Tablet khách hàng.

- Output: request đã điều phối tới đúng service mục tiêu.

4.2 Identity Service

- Chức năng: quản lý đăng nhập, phân quyền và cấp Access Token/Refresh Token.

- Input: Username, Password.

- Output: JWT token, Refresh token, User claims.

4.3 Menu Service

- Chức năng: quản lý thực đơn, danh mục món, giá bán và trạng thái món (còn/hết).

- Input: thông tin món ăn (Mã món, Tên món, Giá, Danh mục, Trạng thái).

- Output: danh sách menu cập nhật cho Order Service và giao diện client.

4.4 Order Service

- Chức năng: nhận và xử lý yêu cầu gọi món, lưu đơn hàng, tính tạm tính.

- Dữ liệu chính: MaMon, SoLuong, GhiChu.

- Luồng xử lý: tạo đơn -> lưu cơ sở dữ liệu -> phát sự kiện OrderCreated.

- Output: OrderId, trạng thái Pending.

4.5 Kitchen Service

- Chức năng: nhận sự kiện từ hàng đợi, xử lý món theo FIFO, cập nhật trạng thái chế biến.

- Input: sự kiện OrderCreated.

- Output: sự kiện OrderPrepared khi món sẵn sàng.

4.6 Payment Service

- Chức năng: tính hóa đơn cuối cùng, xử lý thanh toán và phát hành thông tin giao dịch.

- Input: OrderId, phương thức thanh toán.

- Dữ liệu chính: TongTien, NgayGioThanhToan, MaGiaoDich.

- Output: PaymentSuccess, cập nhật trạng thái bàn về Empty.

4.7 Table Service

- Chức năng: quản lý trạng thái bàn (Empty/Occupied/Reserved), hỗ trợ đổi bàn và đóng bàn sau thanh toán.

- Input: TableId, trạng thái bàn, yêu cầu cập nhật từ Order/Payment Service.

- Output: trạng thái bàn hiện hành phục vụ điều phối khách và vận hành nhà hàng.

4.8 Notification Service

- Chức năng: nhận sự kiện từ Kitchen/Payment và gửi thông báo realtime xuống client qua SignalR/WebSocket.

- Input: OrderPrepared, PaymentSuccess.

- Output: thông báo trạng thái món/đơn theo thời gian thực.

5. Yêu cầu 1d - Mô tả chức năng theo luồng nghiệp vụ

Luồng nghiệp vụ chính của hệ thống:

- Bước 1: Client gửi yêu cầu gọi món qua API Gateway.

- Bước 2: Gateway xác thực và chuyển request sang Order Service.

- Bước 3: Order Service ghi đơn và phát sự kiện OrderCreated lên Event Bus.

- Bước 4: Kitchen Service xử lý món và phát sự kiện OrderPrepared.

- Bước 5: Notification Service đẩy trạng thái realtime cho khách hàng.

- Bước 6: Payment Service chốt hóa đơn, phát PaymentSuccess và hoàn tất quy trình.

Cơ chế bất đồng bộ giúp giảm phụ thuộc giữa các service và tăng khả năng chịu tải cho toàn hệ thống.

Các cơ chế vận hành và độ tin cậy được áp dụng:

- Service Discovery: các service đăng ký vào Service Registry (Consul/Eureka) để định vị động endpoint, giảm phụ thuộc cấu hình tĩnh.

- Fault Tolerance: áp dụng Timeout, Circuit Breaker, Retry giới hạn và Fallback để tránh lỗi dây chuyền khi một service gặp sự cố.

- Data Consistency: sử dụng Eventual Consistency giữa các service qua event; mỗi service quản lý dữ liệu cục bộ và đồng bộ trạng thái theo sự kiện.

- Retry / Dead Letter Queue (DLQ): message lỗi được retry theo backoff; nếu vượt ngưỡng sẽ chuyển vào DLQ để theo dõi và xử lý bù.

6. Yêu cầu 1e - Công nghệ được chọn

Ngăn xếp công nghệ đề xuất:

Lớp (Layer)	Công nghệ / Thư viện	Vai trò
Backend Core	.NET 8 (ASP.NET Core Web API)	Xây dựng microservice hiệu năng cao.
Database	SQL Server 2019	Lưu dữ liệu cấu trúc theo Database-per-Service.
Cache	Redis	Tăng tốc truy vấn và lưu dữ liệu truy cập thường xuyên.
Message Broker	RabbitMQ + MassTransit	Truyền thông điệp bất đồng bộ giữa service.
Communication	gRPC & RESTful API	gRPC cho nội bộ, REST cho client.
Real-time	SignalR	Đẩy cập nhật trạng thái thời gian thực.
Gateway	Ocelot	Điều phối và gom request tại điểm vào.
Container	Docker & Docker Compose	Đóng gói và triển khai đồng nhất môi trường.

7. Kết luận

Qua quá trình phân tích và thiết kế, nhóm đã xây dựng hoàn thiện kiến trúc phần mềm cho hệ thống quản lý nhà hàng dựa trên mô hình Microservices kết hợp Event-Driven Architecture (Kiến trúc hướng sự kiện). Giải pháp này không chỉ đáp ứng đầy đủ các yêu cầu nghiệp vụ phức tạp từ khâu đặt món, chế biến đến thanh toán, mà còn tối ưu hóa được hiệu năng vận hành.

Các kết quả chính đạt được của đồ án bao gồm:

1.

Tính tự chủ và linh hoạt: Việc phân tách hệ thống thành các dịch vụ độc lập (Order, Kitchen, Payment...) giúp từng module có thể phát triển, triển khai và mở rộng (scale) riêng biệt tùy theo tải thực tế, khắc phục nhược điểm cồng kềnh của kiến trúc Monolith.

2.
3.

Khả năng xử lý thời gian thực (Real-time): Việc ứng dụng Message Broker (RabbitMQ) làm trục liên kết và SignalR để đẩy thông báo giúp quy trình gọi món - chế biến diễn ra tức thời, đảm bảo tính đồng bộ dữ liệu cao giữa Bếp và Thu ngân.

4.
5.

Tiêu chuẩn công nghệ hiện đại: Hệ thống được thiết kế trên nền tảng .NET 8, sử dụng Docker để đóng gói và Redis để tăng tốc truy xuất, tạo tiền đề vững chắc cho việc triển khai tự động hóa (CI/CD) và vận hành trên môi trường Cloud sau này.

6.

Tóm lại, giải pháp kiến trúc được đề xuất đảm bảo tính khả thi cao, cân bằng tốt giữa chi phí triển khai và khả năng mở rộng, phù hợp với định hướng phát triển chuỗi nhà hàng thông minh.